

UNIVERSIDAD NACIONAL DE COLOMBIA

PROGRAMACIÓN ORIENTADA A OBJETOS 2026-2S

Actividad 2: Clases y Objetos

Valor 20 %

Estudiante: Alejandro Ramos Rincón

Docente: Walter Hugo Arboleda Mazo

URL del Repositorio en GitHub:

https://github.com/alramosr/Actividad2_P00

Nota: El archivo Readme del repositorio en GitHub cumple con los lineamientos: nombre completo de la universidad, nombre de la actividad, nombre completo del estudiante y nombre completo del docente.

Octubre 2026

Índice

Consideraciones generales	2
1. Ejercicio 2.1: Definición de clases	3
1.1. Código fuente	3
1.2. Diagrama de clases	5
1.3. Ejemplo de ejecución	6
2. Ejercicio 2.2: Definición de atributos con tipos primitivos de datos	7
2.1. Código fuente	7
2.2. Diagrama de clases	10
2.3. Ejemplo de ejecución	11
3. Ejercicio 2.3: Estado de un objeto	12
3.1. Código fuente	12
3.2. Diagrama de clases	17
3.3. Ejemplo de ejecución	18
4. Ejercicio 2.4: Definición de métodos con y sin valores de retorno	19
4.1. Código fuente	19
4.2. Diagrama de clases	26
4.3. Ejemplo de ejecución	27
5. Ejercicio 2.5: Definición de métodos con parámetros	28
5.1. Código fuente	28
5.2. Diagrama de clases	31
5.3. Ejemplo de ejecución	31

Consideraciones generales

Los ejercicios fueron desarrollados en Java a partir del libro *Ejercicios de programación orientada a objetos con Java y UML* de Leonardo Bermón Angarita. Cada ejercicio está organizado en su propio paquete (Ejercicio2_1 a Ejercicio2_5) y cuenta con una clase **Main** que contiene el método **main**. Los valores enumerados se definen dentro de la clase a la que pertenecen, como en el libro.

Todas las clases implementan métodos *get* y *set* para cada uno de sus atributos. Para facilitar la lectura, **en este documento se omiten los métodos *get* y *set***, tanto en el código como en los diagramas de clases; el código completo, con todos sus métodos *get* y *set*, se encuentra en el repositorio de GitHub.

1. Ejercicio 2.1: Definición de clases

Se define la clase `Persona` con sus atributos, un constructor y el método `Imprimir()`. Como ejercicio propuesto se agregan el país de nacimiento (`String`) y el género de la persona (`char` con valores 'H' o 'M'), y se modifican el constructor y el método `Imprimir()` para incluirlos.

1.1. Código fuente

Código 1: `Persona.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_1;
2
3 //Creación de la clase Persona.
4 public class Persona {
5
6     //Sección para declaración de atributos.
7     String Nombre;
8     String Apellidos;
9     String NumeroDocumentoIdentidad;
10    int AnioNacimiento;
11    String PaisNacimiento;
12    char Genero; //'H' para hombre y 'M' para mujer.
13
14    //Creamos el constructor que inicializa los atributos de la persona.
15    public Persona(String Nombre, String Apellidos, String
        NumeroDocumentoIdentidad, int AnioNacimiento, String
        PaisNacimiento, char Genero){
16
17        //Asignamos los valores recibidos a los atributos.
18        this.Nombre = Nombre;
19        this.Apellidos = Apellidos;
20        this.NumeroDocumentoIdentidad = NumeroDocumentoIdentidad;
21        this.AnioNacimiento = AnioNacimiento;
22        this.PaisNacimiento = PaisNacimiento;
23        this.Genero = Genero;
24    }
25
26    //Métodos get y set omitidos en este documento (se encuentran en el
        código fuente del repositorio).
27
28    //Creamos un método para imprimir en pantalla los datos de la
        persona.
29    public void Imprimir(){
30
31        //Impresión de los atributos de la persona.
32        System.out.println("Nombre = " + Nombre);
33        System.out.println("Apellidos = " + Apellidos);
34        System.out.println("Número de documento de identidad = " +
            NumeroDocumentoIdentidad);
35        System.out.println("Año de nacimiento = " + AnioNacimiento);
36        System.out.println("País de nacimiento = " + PaisNacimiento);
37        System.out.println("Género = " + Genero);
38        System.out.println();
39    }
40 }
```

Código 2: Main.java

```
1 package Ejercicio2_1;
2
3 //Creación clase Main.
4 public class Main {
5     public static void main(String[] args){
6
7         //Creación de las dos personas.
8         Persona Persona1 = new Persona("Pedro", "Pérez", "1053121010",
9             1998, "Colombia", 'H');
10        Persona Persona2 = new Persona("Luisa", "León", "1053223344",
11            2001, "Argentina", 'M');
12
13        //Impresión de los datos de la primera persona.
14        Persona1.Imprimir();
15
16        //Impresión de los datos de la segunda persona.
17        Persona2.Imprimir();
18    }
```

1.2. Diagrama de clases

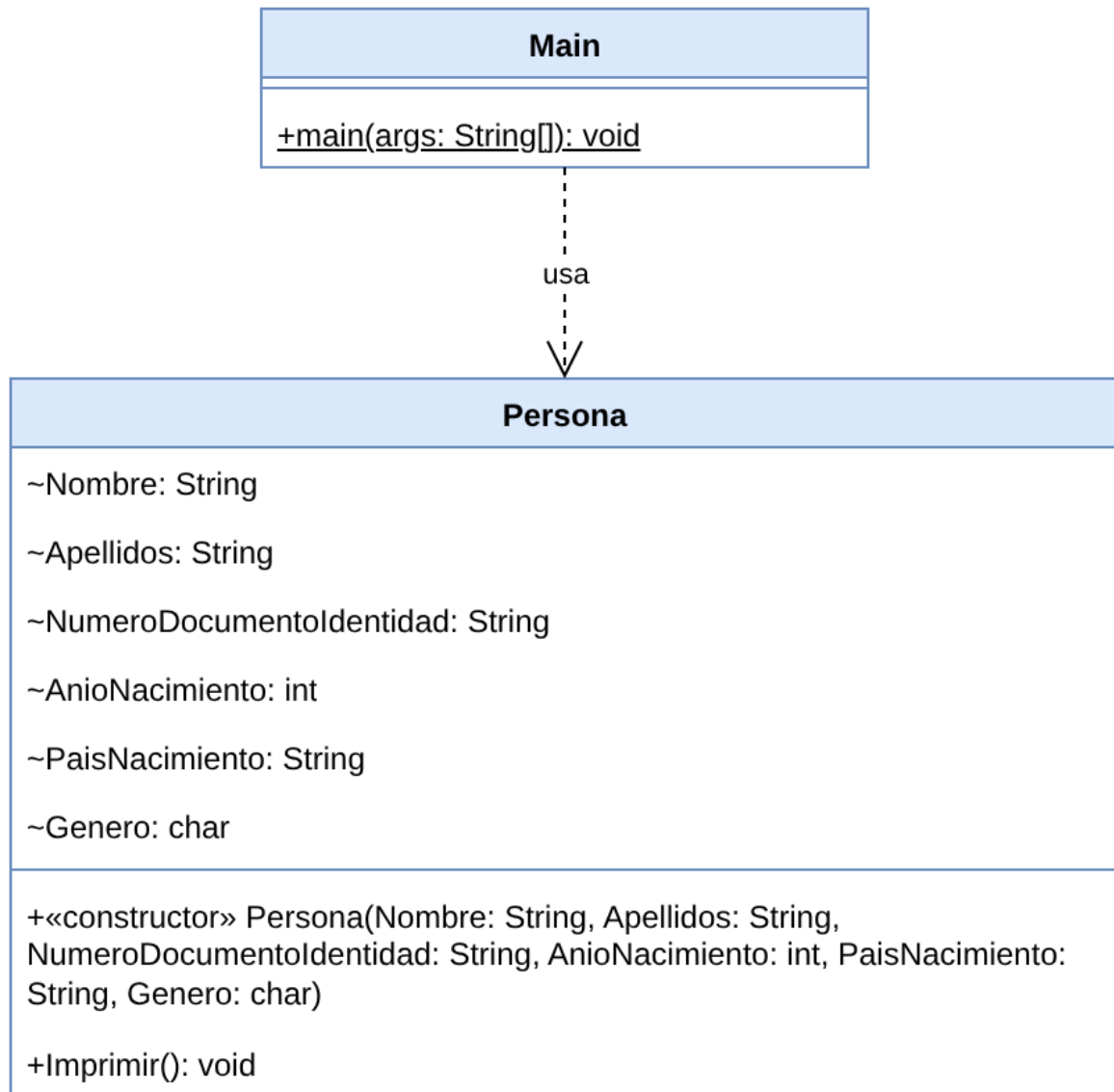


Figura 1: Diagrama de clases del ejercicio 2.1 (sin métodos get y set).

1.3. Ejemplo de ejecución

```
$ java Ejercicio2_1.Main
Nombre = Pedro
Apellidos = Pérez
Número de documento de identidad = 1053121010
Año de nacimiento = 1998
País de nacimiento = Colombia
Género = H

Nombre = Luisa
Apellidos = León
Número de documento de identidad = 1053223344
Año de nacimiento = 2001
País de nacimiento = Argentina
Género = M

$ □
```

Figura 2: Salida en consola de la ejecución del ejercicio 2.1.

2. Ejercicio 2.2: Definición de atributos con tipos primitivos de datos

Se define la clase `Planeta` con atributos de tipos primitivos con sus valores iniciales y el enumerado `TipoPlaneta`, junto con los métodos para imprimir sus datos, calcular su densidad y determinar si es un planeta exterior. Como ejercicio propuesto se agregan el periodo orbital (en años) y el periodo de rotación (en días), y se modifican el constructor y el método `Imprimir()` para incluirlos.

2.1. Código fuente

Código 3: `Planeta.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_2;
2
3 //Creación de la clase Planeta.
4 public class Planeta {
5
6     //Creamos el enumerado con los tipos de planeta de acuerdo con su
7     //tamaño.
8     enum TipoPlaneta {GASEOSO, TERRESTRE, ENANO}
9
10    //Sección para declaración de atributos con sus valores iniciales.
11    String Nombre = null;
12    int CantidadSatelites = 0;
13    double Masa = 0; //En kilogramos.
14    double Volumen = 0; //En kilómetros cúbicos.
15    int Diametro = 0; //En kilómetros.
16    double DistanciaSol = 0; //En millones de kilómetros.
17    TipoPlaneta Tipo;
18    boolean EsObservable = false;
19    double PeriodoOrbital = 0; //En años.
20    double PeriodoRotacion = 0; //En días.
21
22    //Creamos el constructor que inicializa los atributos del planeta.
23    public Planeta(String Nombre, int CantidadSatelites, double Masa,
24        double Volumen, int Diametro, double DistanciaSol, TipoPlaneta
25        Tipo, boolean EsObservable, double PeriodoOrbital, double
26        PeriodoRotacion){
27
28        //Asignamos los valores recibidos a los atributos.
29        this.Nombre = Nombre;
30        this.CantidadSatelites = CantidadSatelites;
31        this.Masa = Masa;
32        this.Volumen = Volumen;
33        this.Diametro = Diametro;
34        this.DistanciaSol = DistanciaSol;
35        this.Tipo = Tipo;
36        this.EsObservable = EsObservable;
37        this.PeriodoOrbital = PeriodoOrbital;
38        this.PeriodoRotacion = PeriodoRotacion;
39    }
40
41    //Métodos get y set omitidos en este documento (se encuentran en el
42    //código fuente del repositorio).
```

```

39 //Creamos un método para imprimir en pantalla los datos del planeta.
40 public void Imprimir(){
41
42     //Impresión de los atributos del planeta.
43     System.out.println("Nombre del planeta = " + Nombre);
44     System.out.println("Cantidad de satélites = " +
45         CantidadSatelites);
46     System.out.println("Masa del planeta = " + Masa);
47     System.out.println("Volumen del planeta = " + Volumen);
48     System.out.println("Diámetro del planeta = " + Diametro);
49     System.out.println("Distancia al sol (millones de km) = " +
50         DistanciaSol);
51     System.out.println("Tipo de planeta = " + Tipo);
52     System.out.println("Es observable = " + EsObservable);
53     System.out.println("Periodo orbital (años) = " + PeriodoOrbital);
54     ;
55     System.out.println("Periodo de rotación (días) = " +
56         PeriodoRotacion);
57 }
58
59 //Creamos un método para calcular la densidad del planeta.
60 public double CalcularDensidad(){
61
62     //Inicializamos una variable para guardar el cálculo de la
63     //densidad (masa / volumen).
64     double Densidad = Masa / Volumen;
65
66     //Retornamos la densidad.
67     return Densidad;
68 }
69
70 //Creamos un método para determinar si el planeta es exterior.
71 public boolean EsPlanetaExterior(){
72
73     //Declaramos la unidad astronómica en millones de kilómetros
74     //(149597870 Km).
75     double UnidadAstronomica = 149.59787;
76
77     //Calculamos dónde termina el cinturón de asteroides (3.4 UA).
78     double Limite = UnidadAstronomica * 3.4;
79
80     //Un planeta exterior está situado más allá del cinturón de
81     //asteroides.
82     boolean EsExterior = DistanciaSol > Limite;
83
84     //Retornamos si el planeta es exterior.
85     return EsExterior;
86 }
87 }

```

Código 4: Main.java

```

1 package Ejercicio2_2;
2
3 //Creación clase Main.
4 public class Main {
5     public static void main(String[] args){
6

```

```

7      //Creación de los dos planetas.
8      Planeta Planeta1 = new Planeta("Tierra", 1, 5.9736E24, 1.08321
9      E12, 12742, 150, Planeta.TipoPlaneta.TERRESTRE, true, 1, 1);
10     Planeta Planeta2 = new Planeta("Júpiter", 79, 1.899E27, 1.4313
11     E15, 139820, 750, Planeta.TipoPlaneta.GASEOSO, true, 11.86,
12     0.41);
13
14     //Impresión de los datos, la densidad y si el primer planeta es
15     exterior.
16     Planeta1.Imprimir();
17     System.out.println("Densidad del planeta = " + Planeta1.
18     CalcularDensidad());
19     System.out.println("Es planeta exterior = " + Planeta1.
20     EsPlanetaExterior());
21     System.out.println();
22
23     //Impresión de los datos, la densidad y si el segundo planeta es
24     exterior.
25     Planeta2.Imprimir();
26     System.out.println("Densidad del planeta = " + Planeta2.
27     CalcularDensidad());
28     System.out.println("Es planeta exterior = " + Planeta2.
29     EsPlanetaExterior());
30
31     }
32 }

```

2.2. Diagrama de clases

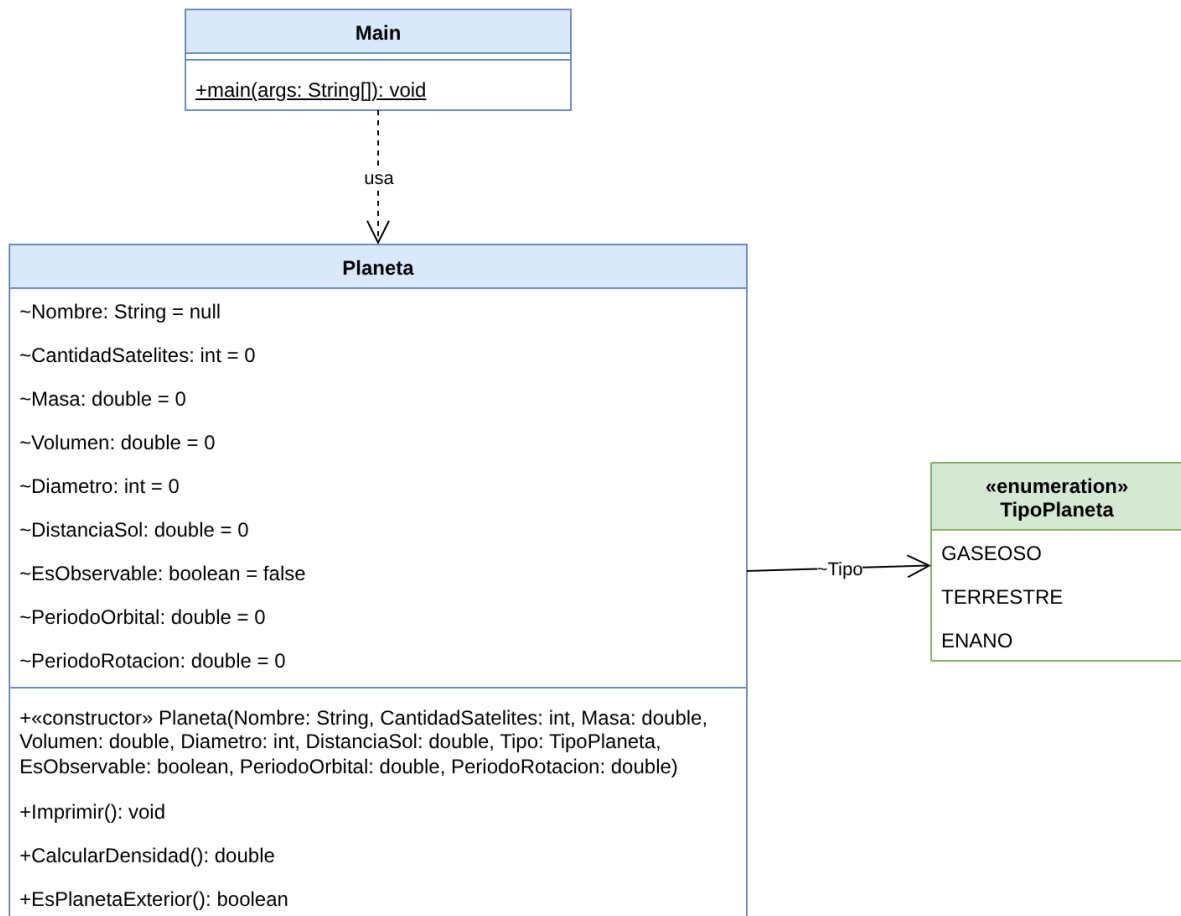


Figura 3: Diagrama de clases del ejercicio 2.2 (sin métodos get y set).

2.3. Ejemplo de ejecución

```
$ java Ejercicio2_2.Main
Nombre del planeta = Tierra
Cantidad de satélites = 1
Masa del planeta = 5.9736E24
Volumen del planeta = 1.08321E12
Diámetro del planeta = 12742
Distancia al sol (millones de km) = 150.0
Tipo de planeta = TERRESTRE
Es observable = true
Periodo orbital (años) = 1.0
Periodo de rotación (días) = 1.0
Densidad del planeta = 5.514720137369484E12
Es planeta exterior = false

Nombre del planeta = Júpiter
Cantidad de satélites = 79
Masa del planeta = 1.899E27
Volumen del planeta = 1.4313E15
Diámetro del planeta = 139820
Distancia al sol (millones de km) = 750.0
Tipo de planeta = GASEOSO
Es observable = true
Periodo orbital (años) = 11.86
Periodo de rotación (días) = 0.41
Densidad del planeta = 1.3267658771745964E12
Es planeta exterior = true
$ □
```

Figura 4: Salida en consola de la ejecución del ejercicio 2.2.

3. Ejercicio 2.3: Estado de un objeto

Se define la clase `Automovil`, cuyo estado cambia mediante sus métodos `get` y `set` y los métodos para acelerar, desacelerar y frenar. Como ejercicio propuesto se agregan el atributo `EsAutomatico` (inicializado en el constructor y con sus métodos `GetEsAutomatico()` y `SetEsAutomatico()`), la multa que se genera cada vez que se intenta superar la velocidad máxima y los métodos para determinar si el automóvil tiene multas y el valor total de sus multas. El método `SetVelocidadActual()` también valida la velocidad y genera una multa si se intenta asignar una velocidad superior a la máxima.

3.1. Código fuente

Código 5: `Automovil.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_3;
2
3 //Creación de la clase Automovil.
4 public class Automovil {
5
6     //Creamos los enumerados para el tipo de combustible, el tipo de
7     //automóvil y el color.
8     enum TipoCom {GASOLINA, BIOETANOL, DIESEL, BIODIESEL, GAS_NATURAL}
9     enum TipoA {CIUDAD, SUBCOMPACTO, COMPACTO, FAMILIAR, EJECUTIVO, SUV}
10    enum TipoColor {BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE, AZUL,
11    VIOLETA}
12
13    //Sección para declaración de atributos.
14    String Marca;
15    int Modelo; //Año de fabricación.
16    int Motor; //Volumen en litros del cilindraje.
17    TipoCom TipoCombustible;
18    TipoA TipoAutomovil;
19    int NumeroPuertas;
20    int CantidadAsientos;
21    int VelocidadMaxima; //En km/h.
22    TipoColor Color;
23    int VelocidadActual = 0; //En km/h.
24    boolean EsAutomatico;
25    int CantidadMultas = 0;
26
27    //Valor de cada multa por intentar superar la velocidad máxima.
28    static final double ValorMulta = 500000;
29
30    //Creamos el constructor que inicializa los atributos del automóvil.
31    public Automovil(String Marca, int Modelo, int Motor, TipoCom
32    TipoCombustible, TipoA TipoAutomovil, int NumeroPuertas, int
33    CantidadAsientos, int VelocidadMaxima, TipoColor Color, boolean
34    EsAutomatico){
35
36        //Asignamos los valores recibidos a los atributos.
37        this.Marca = Marca;
38        this.Modelo = Modelo;
39        this.Motor = Motor;
40        this.TipoCombustible = TipoCombustible;
41        this.TipoAutomovil = TipoAutomovil;
42        this.NumeroPuertas = NumeroPuertas;
```

```

38     this.CantidadAsientos = CantidadAsientos;
39     this.VelocidadMaxima = VelocidadMaxima;
40     this.Color = Color;
41     this.EsAutomatico = EsAutomatico;
42 }
43
44 //Métodos get y set omitidos en este documento (se encuentran en el
    código fuente del repositorio).
45
46 //Creamos un método para acelerar, si se intenta superar la
    velocidad máxima se genera una multa.
47 public void Acelerar(int IncrementoVelocidad){
48
49     //Revisamos que el incremento sea mayor que cero.
50     if (IncrementoVelocidad <= 0){
51         System.out.println("El incremento de velocidad debe ser
            mayor que cero.");
52     }
53
54     //Si el incremento no supera la velocidad máxima, aumentamos la
        velocidad.
55     else if (VelocidadActual + IncrementoVelocidad <=
        VelocidadMaxima){
56         VelocidadActual = VelocidadActual + IncrementoVelocidad;
57     }
58
59     //De otra manera no se puede incrementar la velocidad y se
        genera una multa.
60     else {
61         System.out.println("No se puede incrementar a una velocidad
            superior a la máxima del automóvil.");
62         GenerarMulta();
63     }
64 }
65
66 //Creamos un método para desacelerar sin llegar a una velocidad
    negativa.
67 public void Desacelerar(int DecrementoVelocidad){
68
69     //Revisamos que el decremento sea mayor que cero.
70     if (DecrementoVelocidad <= 0){
71         System.out.println("El decremento de velocidad debe ser
            mayor que cero.");
72     }
73
74     //Si la velocidad no queda negativa, disminuimos la velocidad.
75     else if (VelocidadActual - DecrementoVelocidad >= 0){
76         VelocidadActual = VelocidadActual - DecrementoVelocidad;
77     }
78
79     //De otra manera no se puede decrementar la velocidad.
80     else {
81         System.out.println("No se puede decrementar a una velocidad
            negativa.");
82     }
83 }
84

```

```

85 //Creamos un método para frenar (coloca la velocidad actual en cero)
86 public void Frenar(){
87
88     //Colocamos la velocidad actual en cero.
89     VelocidadActual = 0;
90 }
91
92 //Creamos un método para generar una multa por intentar superar la
    velocidad máxima.
93 public void GenerarMulta(){
94
95     //Aumentamos en uno la cantidad de multas.
96     CantidadMultas = CantidadMultas + 1;
97
98     //Impresión de la multa generada.
99     System.out.println("Se ha generado una multa de $" + ValorMulta
    + ". Multas acumuladas = " + CantidadMultas);
100 }
101
102 //Creamos un método para determinar si el automóvil tiene multas.
103 public boolean TieneMultas(){
104
105     //Inicializamos una variable para guardar si hay al menos una
        multa.
106     boolean TieneMultas = CantidadMultas > 0;
107
108     //Retornamos la variable.
109     return TieneMultas;
110 }
111
112 //Creamos un método para calcular el valor total de las multas.
113 public double CalcularValorTotalMultas(){
114
115     //Inicializamos una variable para guardar el cálculo del valor
        total (cantidad * valor de cada multa).
116     double ValorTotal = CantidadMultas * ValorMulta;
117
118     //Retornamos la variable.
119     return ValorTotal;
120 }
121
122 //Creamos un método para calcular el tiempo de llegada en horas (
    distancia / velocidad actual).
123 public double CalcularTiempoLlegada(int Distancia){
124
125     //Si el automóvil está detenido no se puede calcular el tiempo
        de llegada.
126     if (VelocidadActual == 0){
127         System.out.println("El automóvil está detenido, no es
        posible calcular el tiempo de llegada.");
128
129         //Retornamos -1 para indicar que no se puede calcular.
130         return -1;
131     }
132
133     //Inicializamos una variable para guardar el cálculo del tiempo
        de llegada.

```

```

134         double TiempoLlegada = (double) Distancia / VelocidadActual;
135
136         //Retornamos la variable.
137         return TiempoLlegada;
138     }
139
140     //Creamos un método para imprimir en pantalla los datos del automóvil.
141     public void Imprimir(){
142
143         //Impresión de los atributos del automóvil.
144         System.out.println("Marca = " + Marca);
145         System.out.println("Modelo = " + Modelo);
146         System.out.println("Motor = " + Motor);
147         System.out.println("Tipo de combustible = " + TipoCombustible);
148         System.out.println("Tipo de automóvil = " + TipoAutomovil);
149         System.out.println("Número de puertas = " + NumeroPuertas);
150         System.out.println("Cantidad de asientos = " + CantidadAsientos);
151         ;
152         System.out.println("Velocidad máxima = " + VelocidadMaxima);
153         System.out.println("Color = " + Color);
154         System.out.println("Es automático = " + EsAutomatico);
155     }
}

```

Código 6: Main.java

```

1 package Ejercicio2_3;
2
3 //Creación clase Main.
4 public class Main {
5     public static void main(String[] args){
6
7         //Creación del automóvil.
8         Automovil Auto1 = new Automovil("Ford", 2018, 3, Automovil.
9             TipoCom.DIESEL, Automovil.TipoA.EJECUTIVO, 5, 6, 250,
10             Automovil.TipoColor.NEGRO, true);
11
12         //Impresión de los datos del automóvil.
13         Auto1.Imprimir();
14
15         //Colocamos la velocidad actual en 100 km/h.
16         Auto1.SetVelocidadActual(100);
17         System.out.println("Velocidad actual = " + Auto1.
18             GetVelocidadActual());
19
20         //Aumentamos la velocidad en 20 km/h.
21         Auto1.Acelerar(20);
22         System.out.println("Velocidad actual = " + Auto1.
23             GetVelocidadActual());
24
25         //Disminuimos la velocidad en 50 km/h.
26         Auto1.Desacelerar(50);
27         System.out.println("Velocidad actual = " + Auto1.
28             GetVelocidadActual());
29
30         //Tiempo estimado para recorrer 140 km a la velocidad actual.
31         System.out.println("Tiempo de llegada (horas) = " + Auto1.

```

```

27         CalcularTiempoLlegada(140));
28     //Frenamos el automóvil.
29     Auto1.Frenar();
30     System.out.println("Velocidad actual = " + Auto1.
        GetVelocidadActual());
31
32     //Intento de desacelerar con el automóvil detenido.
33     Auto1.Desacelerar(20);
34
35     //Intentos de acelerar y desacelerar con valores negativos.
36     Auto1.Acelerar(-30);
37     Auto1.Desacelerar(-400);
38     System.out.println("Velocidad actual = " + Auto1.
        GetVelocidadActual());
39
40     //Revisión de multas antes de superar la velocidad máxima.
41     System.out.println("Tiene multas = " + Auto1.TieneMultas());
42
43     //Dos intentos de superar la velocidad máxima (cada uno genera
        una multa).
44     Auto1.Acelerar(300);
45     Auto1.SetVelocidadActual(260);
46
47     //Revisión de multas después de superar la velocidad máxima.
48     System.out.println("Tiene multas = " + Auto1.TieneMultas());
49     System.out.println("Valor total de multas = " + Auto1.
        CalcularValorTotalMultas());
50
51     }
52 }

```

3.2. Diagrama de clases

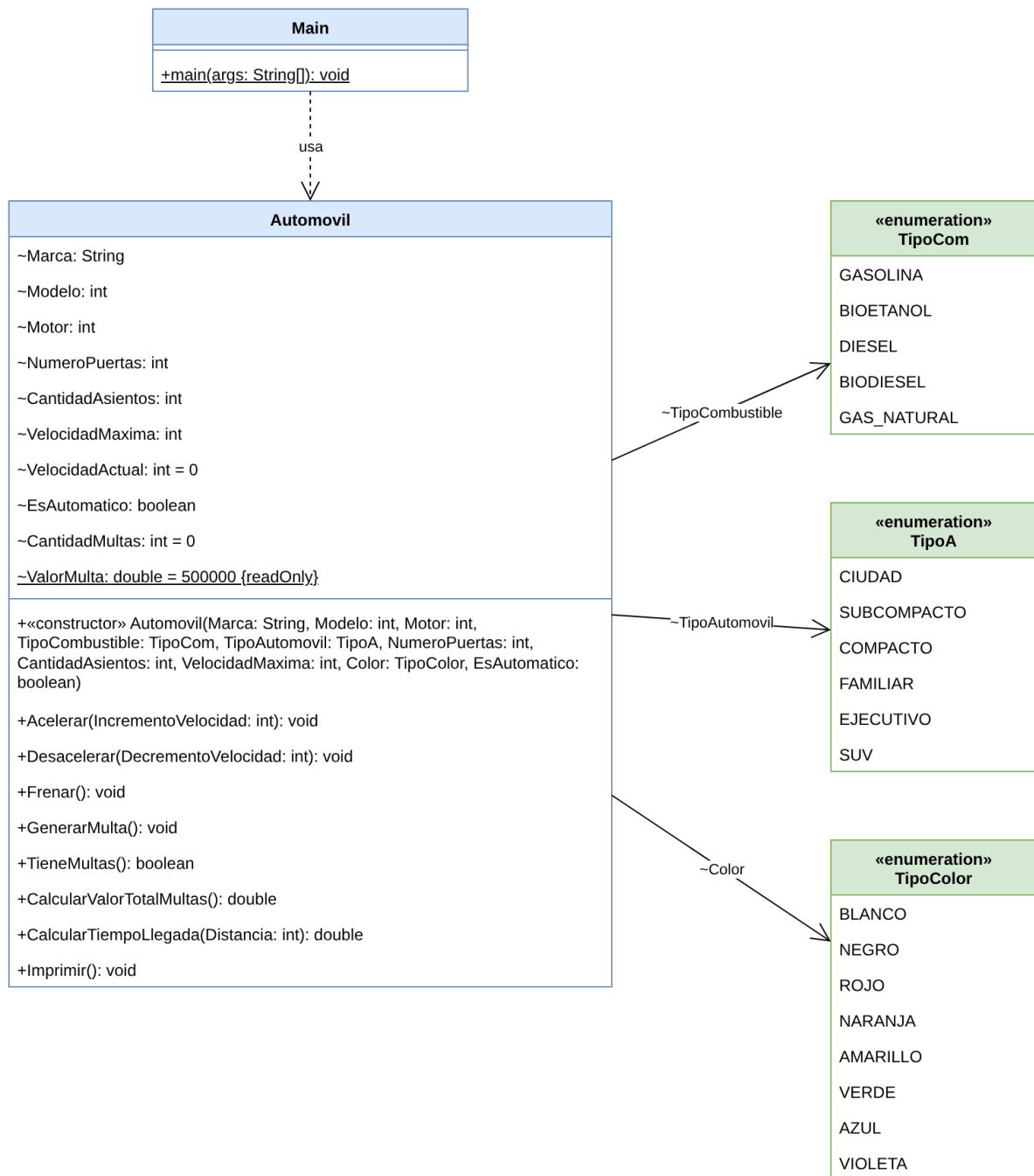


Figura 5: Diagrama de clases del ejercicio 2.3 (sin métodos get y set).

3.3. Ejemplo de ejecución

```
$ java Ejercicio2_3.Main
Marca = Ford
Modelo = 2018
Motor = 3
Tipo de combustible = DIESEL
Tipo de automóvil = EJECUTIVO
Número de puertas = 5
Cantidad de asientos = 6
Velocidad máxima = 250
Color = NEGRO
Es automático = true
Velocidad actual = 100
Velocidad actual = 120
Velocidad actual = 70
Tiempo de llegada (horas) = 2.0
Velocidad actual = 0
No se puede decrementar a una velocidad negativa.
El incremento de velocidad debe ser mayor que cero.
El decremento de velocidad debe ser mayor que cero.
Velocidad actual = 0
Tiene multas = false
No se puede incrementar a una velocidad superior a la máxima del automóvil.
Se ha generado una multa de $500000.0. Multas acumuladas = 1
No se puede asignar una velocidad superior a la máxima del automóvil.
Se ha generado una multa de $500000.0. Multas acumuladas = 2
Tiene multas = true
Valor total de multas = 1000000.0
$ █
```

Figura 6: Salida en consola de la ejecución del ejercicio 2.3.

4. Ejercicio 2.4: Definición de métodos con y sin valores de retorno

Se definen las clases `Circulo`, `Rectangulo`, `Cuadrado` y `TrianguloRectangulo` con métodos con valor de retorno (área, perímetro e hipotenusa) y sin valor de retorno (tipo de triángulo), y una clase de prueba con el método `main`. Como ejercicio propuesto se agregan las clases `Rombo` y `Trapezio` con sus métodos para calcular el área y el perímetro.

4.1. Código fuente

Código 7: `Circulo.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_4;
2
3 //Creación de la clase Circulo.
4 public class Circulo {
5
6     //Sección para declaración de atributos.
7     int Radio; //En centímetros.
8
9     //Creamos el constructor que inicializa el radio del círculo.
10    public Circulo(int Radio){
11
12        //Asignamos el valor recibido al atributo.
13        this.Radio = Radio;
14    }
15
16    //Métodos get y set omitidos en este documento (se encuentran en el
17    //código fuente del repositorio).
18
19    //Creamos un método para calcular el área del círculo.
20    public double CalcularArea(){
21
22        //Inicializamos una variable para guardar el cálculo del área (
23        //pi * radio al cuadrado).
24        double Area = Math.PI * Math.pow(Radio, 2);
25
26        //Retornamos la variable.
27        return Area;
28    }
29
30    //Creamos un método para calcular el perímetro del círculo.
31    public double CalcularPerimetro(){
32
33        //Inicializamos una variable para guardar el cálculo del perí
34        //metro (2 * pi * radio).
35        double Perimetro = 2 * Math.PI * Radio;
36
37        //Retornamos la variable.
38        return Perimetro;
39    }
40 }
```

Código 8: `Rectangulo.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_4;
```

```

2
3 //Creación de la clase Rectangulo.
4 public class Rectangulo {
5
6     //Sección para declaración de atributos.
7     int Base; //En centímetros.
8     int Altura; //En centímetros.
9
10    //Creamos el constructor que inicializa la base y la altura del rect
    ángulo.
11    public Rectangulo(int Base, int Altura){
12
13        //Asignamos los valores recibidos a los atributos.
14        this.Base = Base;
15        this.Altura = Altura;
16    }
17
18    //Métodos get y set omitidos en este documento (se encuentran en el
    código fuente del repositorio).
19
20    //Creamos un método para calcular el área del rectángulo.
21    public double CalcularArea(){
22
23        //Inicializamos una variable para guardar el cálculo del área (
        base * altura).
24        double Area = Base * Altura;
25
26        //Retornamos la variable.
27        return Area;
28    }
29
30    //Creamos un método para calcular el perímetro del rectángulo.
31    public double CalcularPerimetro(){
32
33        //Inicializamos una variable para guardar el cálculo del perí
        metro ((2 * base) + (2 * altura)).
34        double Perimetro = (2 * Base) + (2 * Altura);
35
36        //Retornamos la variable.
37        return Perimetro;
38    }
39 }

```

Código 9: Cuadrado.java (sin métodos get y set)

```

1 package Ejercicio2_4;
2
3 //Creación de la clase Cuadrado.
4 public class Cuadrado {
5
6     //Sección para declaración de atributos.
7     int Lado; //En centímetros.
8
9     //Creamos el constructor que inicializa el lado del cuadrado.
10    public Cuadrado(int Lado){
11
12        //Asignamos el valor recibido al atributo.
13        this.Lado = Lado;

```

```

14     }
15
16     //Métodos get y set omitidos en este documento (se encuentran en el
17     código fuente del repositorio).
18
19     //Creamos un método para calcular el área del cuadrado.
20     public double CalcularArea(){
21
22         //Inicializamos una variable para guardar el cálculo del área (
23         lado * lado).
24         double Area = Lado * Lado;
25
26         //Retornamos la variable.
27         return Area;
28     }
29
30     //Creamos un método para calcular el perímetro del cuadrado.
31     public double CalcularPerimetro(){
32
33         //Inicializamos una variable para guardar el cálculo del perí
34         metro (4 * lado).
35         double Perimetro = 4 * Lado;
36
37         //Retornamos la variable.
38         return Perimetro;
39     }
40 }

```

Código 10: TrianguloRectangulo.java (sin métodos get y set)

```

1 package Ejercicio2_4;
2
3 //Creación de la clase TrianguloRectangulo.
4 public class TrianguloRectangulo {
5
6     //Sección para declaración de atributos.
7     int Base; //En centímetros.
8     int Altura; //En centímetros.
9
10    //Margen de error para comparar los lados con la hipotenusa (que no
11    siempre es un número entero).
12    static final double Tolerancia = 0.000000001;
13
14    //Creamos el constructor que inicializa la base y la altura del triá
15    ngulo.
16    public TrianguloRectangulo(int Base, int Altura){
17
18        //Asignamos los valores recibidos a los atributos.
19        this.Base = Base;
20        this.Altura = Altura;
21    }
22
23    //Métodos get y set omitidos en este documento (se encuentran en el
24    código fuente del repositorio).
25
26    //Creamos un método para calcular el área del triángulo.
27    public double CalcularArea(){
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

26      //Inicializamos una variable para guardar el cálculo del área (
          base * altura / 2).
27      double Area = (Base * Altura) / 2.0;
28
29      //Retornamos la variable.
30      return Area;
31  }
32
33      //Creamos un método para calcular el perímetro del triángulo.
34      public double CalcularPerimetro(){
35
36          //Inicializamos una variable para guardar el cálculo del perí
              metro (base + altura + hipotenusa).
37          double Perimetro = Base + Altura + CalcularHipotenusa();
38
39          //Retornamos la variable.
40          return Perimetro;
41      }
42
43      //Creamos un método para calcular la hipotenusa con el teorema de
          Pitágoras.
44      public double CalcularHipotenusa(){
45
46          //Inicializamos una variable para guardar el cálculo de la
              hipotenusa (raíz de base al cuadrado + altura al cuadrado).
47          double Hipotenusa = Math.sqrt(Math.pow(Base, 2) + Math.pow(
              Altura, 2));
48
49          //Retornamos la variable.
50          return Hipotenusa;
51      }
52
53      //Creamos un método para determinar qué tipo de triángulo es (no
          retorna ningún valor).
54      public void DeterminarTipoTriangulo(){
55
56          //Calculamos la hipotenusa, que es el tercer lado del triángulo.
57          double Hipotenusa = CalcularHipotenusa();
58
59          //Comparamos los lados entre sí teniendo en cuenta la tolerancia
              .
60          boolean BaseIgualAltura = Math.abs(Base - Altura) < Tolerancia;
61          boolean BaseIgualHipotenusa = Math.abs(Base - Hipotenusa) <
              Tolerancia;
62          boolean AlturaIgualHipotenusa = Math.abs(Altura - Hipotenusa) <
              Tolerancia;
63
64          //Equilátero: todos sus lados son iguales.
65          if (BaseIgualAltura && BaseIgualHipotenusa &&
              AlturaIgualHipotenusa){
66              System.out.println("Es un triángulo equilátero");
67          }
68
69          //Escaleno: todos sus lados son diferentes.
70          else if (!BaseIgualAltura && !BaseIgualHipotenusa && !
              AlturaIgualHipotenusa){
71              System.out.println("Es un triángulo escaleno");
72          }

```

```

73
74     //Isósceles: tiene dos lados iguales.
75     else {
76         System.out.println("Es un triángulo isósceles");
77     }
78 }
79 }

```

Código 11: Rombo.java (sin métodos get y set)

```

1  package Ejercicio2_4;
2
3  //Creación de la clase Rombo.
4  public class Rombo {
5
6      //Sección para declaración de atributos.
7      int DiagonalMayor; //En centímetros.
8      int DiagonalMenor; //En centímetros.
9
10     //Creamos el constructor que inicializa las diagonales del rombo.
11     public Rombo(int DiagonalMayor, int DiagonalMenor){
12
13         //Asignamos los valores recibidos a los atributos.
14         this.DiagonalMayor = DiagonalMayor;
15         this.DiagonalMenor = DiagonalMenor;
16     }
17
18     //Métodos get y set omitidos en este documento (se encuentran en el
19     //código fuente del repositorio).
20
21     //Creamos un método para calcular el área del rombo.
22     public double CalcularArea(){
23
24         //Inicializamos una variable para guardar el cálculo del área (
25         //diagonal mayor * diagonal menor / 2).
26         double Area = (DiagonalMayor * DiagonalMenor) / 2.0;
27
28         //Retornamos la variable.
29         return Area;
30     }
31
32     //Creamos un método para calcular el lado del rombo con el teorema
33     //de Pitágoras (las diagonales se cortan por la mitad).
34     public double CalcularLado(){
35
36         //Inicializamos una variable para guardar el cálculo del lado.
37         double Lado = Math.sqrt(Math.pow(DiagonalMayor / 2.0, 2) + Math.
38         pow(DiagonalMenor / 2.0, 2));
39
40         //Retornamos la variable.
41         return Lado;
42     }
43
44     //Creamos un método para calcular el perímetro del rombo.
45     public double CalcularPerimetro(){
46
47         //Inicializamos una variable para guardar el cálculo del perí
48         //metro (4 * lado).

```

```

44         double Perimetro = 4 * CalcularLado();
45
46         //Retornamos la variable.
47         return Perimetro;
48     }
49 }

```

Código 12: Trapecio.java (sin métodos get y set)

```

1  package Ejercicio2_4;
2
3  //Creación de la clase Trapecio.
4  public class Trapecio {
5
6      //Sección para declaración de atributos.
7      int BaseMayor; //En centímetros.
8      int BaseMenor; //En centímetros.
9      int Altura; //En centímetros.
10     int LadoIzquierdo; //Lado no paralelo, en centímetros.
11     int LadoDerecho; //Lado no paralelo, en centímetros.
12
13     //Creamos el constructor que inicializa las bases, la altura y los
14     //lados del trapecio.
15     public Trapecio(int BaseMayor, int BaseMenor, int Altura, int
16         LadoIzquierdo, int LadoDerecho){
17
18         //Asignamos los valores recibidos a los atributos.
19         this.BaseMayor = BaseMayor;
20         this.BaseMenor = BaseMenor;
21         this.Altura = Altura;
22         this.LadoIzquierdo = LadoIzquierdo;
23         this.LadoDerecho = LadoDerecho;
24     }
25
26     //Métodos get y set omitidos en este documento (se encuentran en el
27     //código fuente del repositorio).
28
29     //Creamos un método para calcular el área del trapecio.
30     public double CalcularArea(){
31
32         //Inicializamos una variable para guardar el cálculo del área ((
33         //base mayor + base menor) * altura / 2).
34         double Area = ((BaseMayor + BaseMenor) * Altura) / 2.0;
35
36         //Retornamos la variable.
37         return Area;
38     }
39
40     //Creamos un método para calcular el perímetro del trapecio.
41     public double CalcularPerimetro(){
42
43         //Inicializamos una variable para guardar el cálculo del perí
44         //metro (suma de sus cuatro lados).
45         double Perimetro = BaseMayor + BaseMenor + LadoIzquierdo +
46             LadoDerecho;
47
48         //Retornamos la variable.
49         return Perimetro;

```

```

44     }
45 }

```

Código 13: Main.java

```

1  package Ejercicio2_4;
2
3  //Creación clase Main.
4  public class Main {
5      public static void main(String[] args){
6
7          //Creación de las figuras geométricas.
8          Circulo Figura1 = new Circulo(2);
9          Rectangulo Figura2 = new Rectangulo(1, 2);
10         Cuadrado Figura3 = new Cuadrado(3);
11         TrianguloRectangulo Figura4 = new TrianguloRectangulo(3, 5);
12         Rombo Figura5 = new Rombo(8, 6);
13         Trapecio Figura6 = new Trapecio(10, 4, 4, 5, 5);
14
15         //Impresión del área y el perímetro del círculo.
16         System.out.println("El área del círculo es = " + Figura1.
17             CalcularArea());
18         System.out.println("El perímetro del círculo es = " + Figura1.
19             CalcularPerimetro());
20         System.out.println();
21
22         //Impresión del área y el perímetro del rectángulo.
23         System.out.println("El área del rectángulo es = " + Figura2.
24             CalcularArea());
25         System.out.println("El perímetro del rectángulo es = " + Figura2.
26             CalcularPerimetro());
27         System.out.println();
28
29         //Impresión del área y el perímetro del cuadrado.
30         System.out.println("El área del cuadrado es = " + Figura3.
31             CalcularArea());
32         System.out.println("El perímetro del cuadrado es = " + Figura3.
33             CalcularPerimetro());
34         System.out.println();
35
36         //Impresión del área, el perímetro, la hipotenusa y el tipo del
37         // triángulo rectángulo.
38         System.out.println("El área del triángulo es = " + Figura4.
39             CalcularArea());
40         System.out.println("El perímetro del triángulo es = " + Figura4.
41             CalcularPerimetro());
42         System.out.println("La hipotenusa del triángulo es = " + Figura4.
43             CalcularHipotenusa());
44         Figura4.DeterminarTipoTriangulo();
45         System.out.println();
46
47         //Impresión del área y el perímetro del rombo.
48         System.out.println("El área del rombo es = " + Figura5.
49             CalcularArea());
50         System.out.println("El perímetro del rombo es = " + Figura5.
51             CalcularPerimetro());
52         System.out.println();
53     }
54 }

```

```

42 //Impresión del área y el perímetro del trapecio.
43 System.out.println("El área del trapecio es = " + Figura6.
    CalcularArea());
44 System.out.println("El perímetro del trapecio es = " + Figura6.
    CalcularPerimetro());
45
46 }
47 }

```

4.2. Diagrama de clases

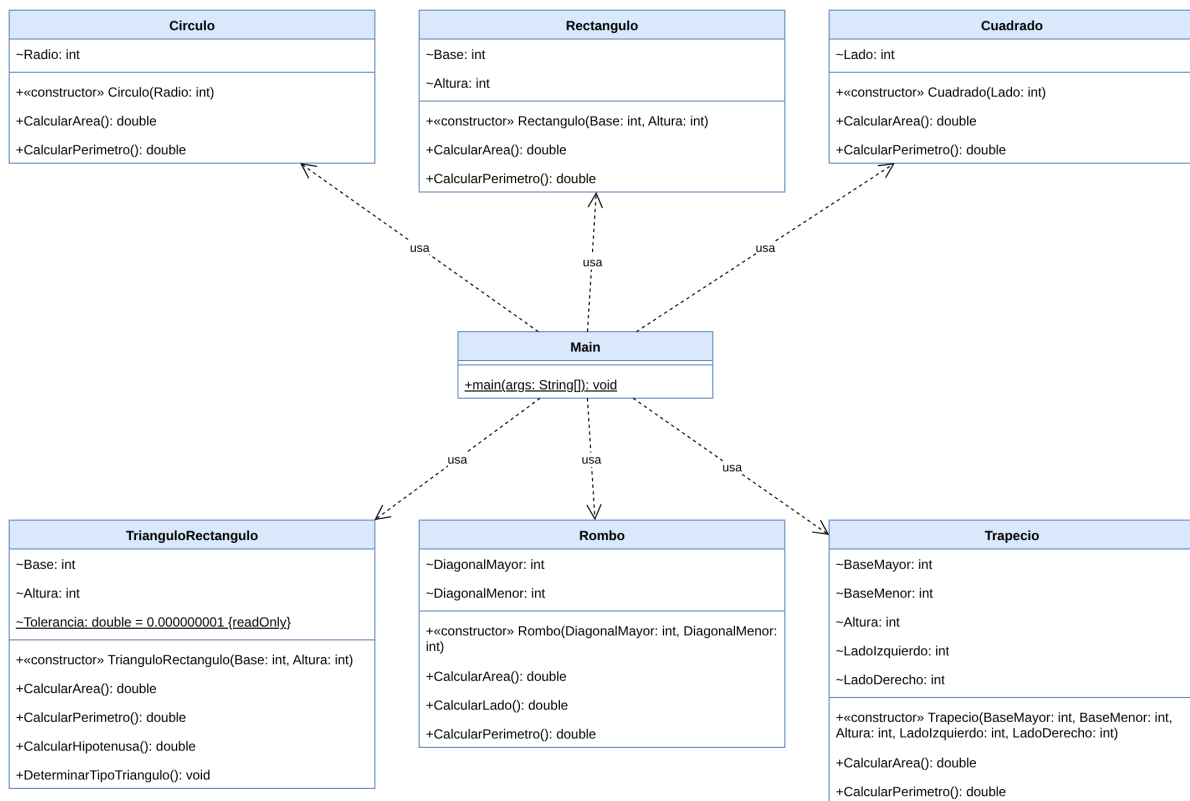


Figura 7: Diagrama de clases del ejercicio 2.4 (sin métodos get y set).

4.3. Ejemplo de ejecución

```
$ java Ejercicio2_4.Main
El área del círculo es = 12.566370614359172
El perímetro del círculo es = 12.566370614359172

El área del rectángulo es = 2.0
El perímetro del rectángulo es = 6.0

El área del cuadrado es = 9.0
El perímetro del cuadrado es = 12.0

El área del triángulo es = 7.5
El perímetro del triángulo es = 13.8309518948453
La hipotenusa del triángulo es = 5.830951894845301
Es un triángulo escaleno

El área del rombo es = 24.0
El perímetro del rombo es = 20.0

El área del trapecio es = 28.0
El perímetro del trapecio es = 24.0
$
```

Figura 8: Salida en consola de la ejecución del ejercicio 2.4.

5. Ejercicio 2.5: Definición de métodos con parámetros

Se define la clase `CuentaBancaria` con métodos que reciben parámetros para consignar y retirar dinero. Como ejercicio propuesto se agregan el atributo con el porcentaje de interés mensual y un método que calcula el nuevo saldo aplicando dicho interés.

5.1. Código fuente

Código 14: `CuentaBancaria.java` (sin métodos `get` y `set`)

```
1 package Ejercicio2_5;
2
3 //Creación de la clase CuentaBancaria.
4 public class CuentaBancaria {
5
6     //Creamos el enumerado con los tipos de cuenta bancaria.
7     enum Tipo {AHORROS, CORRIENTE}
8
9     //Sección para declaración de atributos.
10    String NombresTitular;
11    String ApellidosTitular;
12    int NumeroCuenta;
13    Tipo TipoCuenta;
14    double Saldo = 0; //El saldo inicial es cero.
15    double PorcentajeInteresMensual; //Por ejemplo 1.5 significa 1.5 %
        mensual.
16
17    //Creamos el constructor que inicializa los atributos (el saldo no
        se pasa porque inicia en cero).
18    public CuentaBancaria(String NombresTitular, String ApellidosTitular
        , int NumeroCuenta, Tipo TipoCuenta, double
        PorcentajeInteresMensual){
19
20        //Asignamos los valores recibidos a los atributos.
21        this.NombresTitular = NombresTitular;
22        this.ApellidosTitular = ApellidosTitular;
23        this.NumeroCuenta = NumeroCuenta;
24        this.TipoCuenta = TipoCuenta;
25        this.PorcentajeInteresMensual = PorcentajeInteresMensual;
26    }
27
28    //Métodos get y set omitidos en este documento (se encuentran en el
        código fuente del repositorio).
29
30    //Creamos un método para imprimir en pantalla los datos de la cuenta
        bancaria.
31    public void Imprimir(){
32
33        //Impresión de los atributos de la cuenta bancaria.
34        System.out.println("Nombres del titular = " + NombresTitular);
35        System.out.println("Apellidos del titular = " + ApellidosTitular
        );
36        System.out.println("Número de cuenta = " + NumeroCuenta);
37        System.out.println("Tipo de cuenta = " + TipoCuenta);
38        System.out.println("Saldo = " + Saldo);
39        System.out.println("Porcentaje de interés mensual = " +
        PorcentajeInteresMensual + " %");
```

```

40 }
41
42 //Creamos un método para imprimir en pantalla el saldo actual.
43 public void ConsultarSaldo(){
44
45     //Impresión del saldo actual.
46     System.out.println("El saldo actual es = " + Saldo);
47 }
48
49 //Creamos un método para consignar un valor en la cuenta (el valor
    debe ser mayor que cero).
50 public boolean Consignar(double Valor){
51
52     //Revisamos que el valor a consignar sea mayor que cero.
53     if (Valor <= 0){
54         System.out.println("El valor a consignar debe ser mayor que
            cero.");
55
56         //Retornamos falso porque no se pudo consignar.
57         return false;
58     }
59
60     //Actualizamos el saldo con el valor consignado.
61     Saldo = Saldo + Valor;
62     System.out.println("Se ha consignado $" + Valor + " en la cuenta
        . El nuevo saldo es $" + Saldo);
63
64     //Retornamos verdadero porque se pudo consignar.
65     return true;
66 }
67
68 //Creamos un método para retirar un valor de la cuenta (no puede
    superar el saldo actual).
69 public boolean Retirar(double Valor){
70
71     //Revisamos que el valor a retirar sea mayor que cero.
72     if (Valor <= 0){
73         System.out.println("El valor a retirar debe ser mayor que
            cero.");
74
75         //Retornamos falso porque no se pudo retirar.
76         return false;
77     }
78
79     //Revisamos que el valor a retirar no supere el saldo actual.
80     if (Valor > Saldo){
81         System.out.println("No se puede retirar $" + Valor + "
            porque supera el saldo actual de $" + Saldo);
82
83         //Retornamos falso porque no se pudo retirar.
84         return false;
85     }
86
87     //Actualizamos el saldo con el valor retirado.
88     Saldo = Saldo - Valor;
89     System.out.println("Se ha retirado $" + Valor + " de la cuenta.
        El nuevo saldo es $" + Saldo);
90

```

```

91         //Retornamos verdadero porque se pudo retirar.
92         return true;
93     }
94
95     //Creamos un método para calcular el nuevo saldo aplicando el interés
96     //mensual.
97     public double AplicarInteresMensual(){
98
99         //Inicializamos una variable para guardar el cálculo del interés
100         // (saldo * porcentaje / 100).
101         double Interes = Saldo * PorcentajeInteresMensual / 100;
102
103         //Actualizamos el saldo sumándole el interés.
104         Saldo = Saldo + Interes;
105         System.out.println("Se aplicó un interés de $" + Interes + ". El
106         nuevo saldo es $" + Saldo);
107
108         //Retornamos el nuevo saldo.
109         return Saldo;
110     }
111 }

```

Código 15: Main.java

```

1 package Ejercicio2_5;
2
3 //Creación clase Main.
4 public class Main {
5     public static void main(String[] args){
6
7         //Creación de la cuenta bancaria.
8         CuentaBancaria Cuenta = new CuentaBancaria("Pedro", "Pérez",
9         123456789, CuentaBancaria.Tipo.AHORROS, 1.5);
10
11         //Impresión de los datos de la cuenta bancaria.
12         Cuenta.Imprimir();
13
14         //Operaciones de consignar y retirar sobre la cuenta.
15         Cuenta.Consignar(200000);
16         Cuenta.Consignar(300000);
17         Cuenta.Retirar(400000);
18
19         //Consulta del saldo actual.
20         Cuenta.ConsultarSaldo();
21
22         //Intento de retirar más dinero del que hay en la cuenta.
23         Cuenta.Retirar(500000);
24
25         //Aplicación del interés mensual sobre el saldo.
26         Cuenta.AplicarInteresMensual();
27     }
28 }

```

5.2. Diagrama de clases

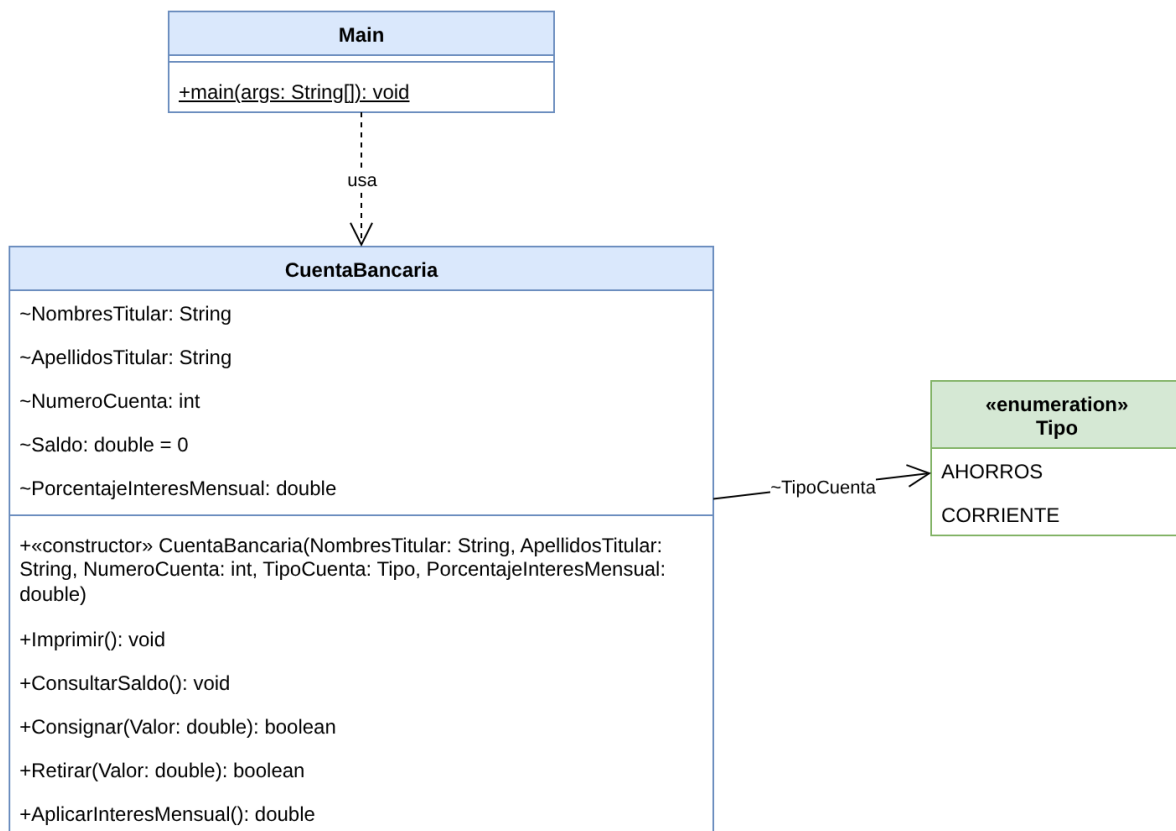


Figura 9: Diagrama de clases del ejercicio 2.5 (sin métodos get y set).

5.3. Ejemplo de ejecución

```
$ java Ejercicio2_5.Main
Nombres del titular = Pedro
Apellidos del titular = Pérez
Número de cuenta = 123456789
Tipo de cuenta = AHORROS
Saldo = 0.0
Porcentaje de interés mensual = 1.5 %
Se ha consignado $200000.0 en la cuenta. El nuevo saldo es $200000.0
Se ha consignado $300000.0 en la cuenta. El nuevo saldo es $500000.0
Se ha retirado $400000.0 de la cuenta. El nuevo saldo es $100000.0
El saldo actual es = 100000.0
No se puede retirar $500000.0 porque supera el saldo actual de $100000.0
Se aplicó un interés de $1500.0. El nuevo saldo es $101500.0
$
```

Figura 10: Salida en consola de la ejecución del ejercicio 2.5.